SISTEMAS DISTRIBUIDOS · LAB S11

# Mexi Banco: una verdad, varias copias

El dinero se queda en PostgreSQL. Los avisos se van a MongoDB y el saldo que se muestra se copia en Redis.

Sábado 31 de octubre de 2026 · laboratorio de 70 minutos · en equipo, entrega individual · **entrega: sábado 7 de noviembre, antes de las 07:00**

**Cómo se trabaja hoy**

Siéntense con su **equipo del proyecto**. Apóyense: repártanse la investigación, comparen pantallas, ayúdense a depurar. Pero **cada quien** hace los pasos en su máquina, toma sus propias capturas y **entrega lo suyo**: su código y su bitácora. Las casillas ☐ de cada sección sirven para que lleves la cuenta; se guardan solo en tu navegador.

**Antes de clase, en tu casa (15 a 25 minutos)**

Descarga la rama de hoy y construye todo una vez. La primera vez baja las imágenes de Redis y MongoDB y las dependencias nuevas, y tarda varios minutos. Es la **solución del lab S09** (la mensajería con RabbitMQ) más Redis y MongoDB ya instalados, pero sin usar.

```
# 1. Descarga la rama base del lab en una carpeta nueva y crea tu propia rama de trabajo
git clone --branch 06-poliglota-base https://github.com/OscarRuiz21/mexi-banco.git mexi-banco-s11
cd mexi-banco-s11
git switch -c trabajo-s11

# 2. Construye las imágenes y levanta todo en segundo plano (la primera vez tarda)
docker compose up --build -d
docker compose ps        # muestra cada contenedor; espera a que todos digan "healthy"

# 3. Apaga todo hasta mañana (sin -v: conserva los datos)
docker compose down
```

**Windows:** corre esto en Git Bash (ver el recuadro de abajo).

Si ya tienes el repo de otro lab: `git fetch origin` (trae las ramas nuevas) y `git switch 06-poliglota-base` (cámbiate a la de hoy). Abre la carpeta en IntelliJ o VS Code: en clase vas a editar dos servicios, `cuenta` y `notificacion`.

**¿Usas Windows? Usa Git Bash, no PowerShell**

Todos los comandos de esta guía son de **bash**, la terminal de Mac y Linux. En Windows, córrelos en **Git Bash**, que se instaló junto con Git (búscalo en el menú Inicio). En **PowerShell** y en **CMD** no funcionan: ahí `curl` es otro comando, las comillas del JSON se rompen y no existen `grep`, `awk` ni `seq`.

- **Dentro del IDE:** en VS Code, abre la flechita junto al **+** de la terminal y elige **Git Bash**. En IntelliJ, ve a **Settings → Tools → Terminal** y en *Shell path* pon `C:\Program Files\Git\bin\bash.exe`; luego abre una terminal nueva.
- **Si ya usas WSL** (la terminal de Ubuntu dentro de Windows), también sirve: es Linux de verdad. Revisa que en Docker Desktop esté activo *Settings → Resources → WSL integration* para tu Ubuntu.
- **Docker Desktop tiene que estar abierto** mientras trabajas.

Por eso todos los `docker compose exec` de la guía llevan `-T`: le dice a Docker que no abra una terminal interactiva. En Mac y Linux da igual; en Git Bash evita el error `the input device is not a TTY`.

# 1 Por qué: ninguna base sirve para todo

Minutos 0 a 10 · se platica en grupo; tú solo lees.

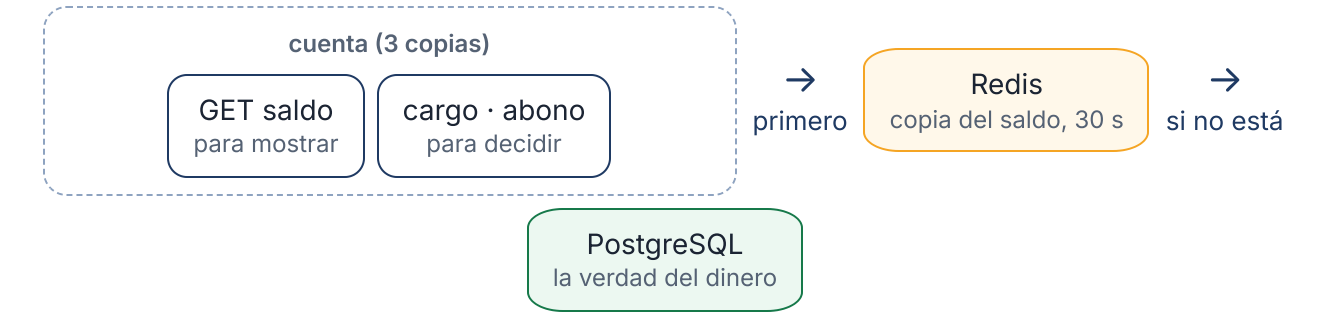
## Cómo está hoy Mexi Banco

Después de la S09, **todo** vive en PostgreSQL: las cuentas, los movimientos, las transferencias, el outbox y también los avisos que manda notificación. Funciona, pero hay dos datos a los que Postgres no les queda bien:

1. **El saldo que el cliente consulta.** La app lo pide cada vez que alguien abre la pantalla: muchas más veces de las que cambia. Cada consulta es una lectura a Postgres, la misma base que tiene que aguantar los cargos y los abonos.
2. **Los avisos.** Un SMS lleva teléfono y texto, un correo lleva dirección, asunto y cuerpo, un push lleva el token del celular. En una tabla eso es una columna por campo de cualquier canal, casi todas vacías, o una tabla por canal. Y nunca se actualizan: se escriben una vez y se leen enteros.

## La herramienta correcta para cada dato

En clase dijimos que una base de datos es como una herramienta: el martillo sirve para clavos, no para tornillos. Hoy Mexi Banco usa **tres**:




El GET pregunta primero a Redis. Los cargos y abonos siempre leen y escriben en Postgres.




Los avisos dejan la bandeja de Postgres y se van a MongoDB. Lo naranja es lo que vas a construir.

## Verdad contra copia

La pregunta del día no es "¿SQL o NoSQL?", sino **qué dato es la verdad y qué dato es una copia**. La verdad tiene que estar en un lugar que no la pierda ni la contradiga. Una copia se puede reconstruir, y por eso se vale guardarla en otro motor, con otra forma y hasta un poco atrasada.

| DATO | DÓNDE VIVE | ¿VERDAD O COPIA? | CÓMO SE MANTIENE |
|---|---|---|---|
| Cuenta y saldo | PostgreSQL | verdad | Transacción local con `@Version` |
| Movimientos | PostgreSQL | verdad | Asientos con Idempotency-Key (S08) |
| Transferencias y outbox | PostgreSQL | verdad | Saga y Outbox (S08 y S09) |
| Saldo **para mostrar** | **Redis** | copia hasta 30 s atrás | Se borra al cambiar el saldo, y caduca sola |
| Avisos enviados | **MongoDB** | verdad de la bitácora de avisos | Los escribe el consumidor de la S09 |

**Las tres reglas de hoy**

1. **Nada que sea dinero cambia de motor.** Saldo, movimientos y transferencias se quedan en Postgres.
2. **La caché muestra, nunca decide.** El cargo que revisa si alcanza el saldo lee Postgres, siempre.
3. **No escribas lo mismo en dos bases.** Sin una transacción común, tarde o temprano una se queda sin el dato. Es el problema que vimos en la S09.

## 2 El mapa del código

Antes de tocar nada, ubica los archivos. Todas las rutas son relativas a la carpeta del repo.

Los huecos que llenas están marcados con `// TODO S11:`. Para verlos todos: en IntelliJ abre **View → Tool Windows → TODO**; en VS Code, la búsqueda en archivos (**Ctrl+Shift+F** o **Cmd+Shift+F**) con el texto `TODO S11`.

### Lo que vas a editar

`cuenta/src/main/java/mx/mexibanco/cuenta/ConfiguracionCache.java`

Enciende la caché de Spring y configura la caché `cuentas`: cuánto vive cada entrada, que se guarde en JSON y que el borrado espere al commit.

Parte 1

`cuenta/src/main/java/mx/mexibanco/cuenta/ConsultaDeSaldo.java`

La consulta del saldo **para mostrar** ( `GET /cuentas/{clabe}` ). Es la única que puede contestar desde Redis.

Parte 1

`cuenta/src/main/java/mx/mexibanco/cuenta/CuentaService.java`

Las reglas de la cuenta: `cargar` y `abonar` cambian el saldo. Ahí se borra la copia de Redis.

Parte 1

`notificacion/src/main/java/mx/mexibanco/notificacion/avisos/BitacoraDeAvisos.java`

Guarda cada aviso como documento en MongoDB, crea el índice y lee los últimos 10 de una CLABE.

Parte 2

`notificacion/src/main/java/mx/mexibanco/notificacion/mensajeria/ConsumidorDeTransferencias.java`

El consumidor de RabbitMQ de la S09. Cambias **una línea** para que guarde en MongoDB en lugar de Postgres.

Parte 2

## Lo que ya está listo (solo léelo)

| ARCHIVO | QUÉ HACE |
|---|---|
| `CuentaVista.java`<br>cuenta | Lo que contesta el GET del saldo: `clabe` , `titular` , `saldo` y `leidaEn` , la hora en que se leyó de Postgres. Si la respuesta sale de la caché, `leidaEn` es vieja: así ves la edad del dato sin abrir Redis. |
| `CuentaController.java`<br>cuenta | Ya manda el GET a `ConsultaDeSaldo` y los cargos y abonos a `CuentaService` . |
| `application.yml`<br>cuenta/src/main/resources | Dónde está Redis ( `spring.data.redis` ), el TTL de 30 s ( `mexibanco.cache.ttl` ) y `spring.cache.type` , que con `CACHE_TIPO=none` apaga la caché sin tocar código. |
| `Contacto.java`<br>notificacion/.../avisos | Los tres canales: `Sms(telefono)` , `Correo(correo)` y `Push(token)` . Es una interfaz sellada: un `switch` que olvide un canal no compila. |
| `DirectorioDeContactos.java`<br>notificacion/.../avisos | Quién recibe qué: Ana ( `...001` ) pidió SMS, Beto ( `...002` ) pidió correo y cualquier otra cuenta, como Carla, recibe un push. |

| | |
|---|---|
| `NotificacionController.java`<br>notificacion | Ya expone `GET /notificaciones/{clabe}`, que llama a `bitacora.ultimos(clabe)`. |
| `docker-compose.yml`<br>raíz del repo | Trae los servicios `redis` y `mongo`. Redis guarda solo en memoria, con 64 MB y desalojo `allkeys-lru`. |

# 3 Levantar y ver el punto de partida

Minutos 10 a 15 · Parte 0 · antes de cambiar código, mide cómo está.

## 3.1 · Levanta el banco

```
cd mexi-banco-s11
docker compose up -d     # levanta lo que construiste en casa
docker compose ps        # trece contenedores "healthy": los de la S09 más redis y mongo
```

Si tu laptop sufre, este lab no usa `spei`: apágalo con `docker compose stop spei`. Deja las **tres** copias de `cuenta`: así vas a ver que la caché la comparten. Si el puerto 8080 está ocupado, levanta con `PUERTO_GATEWAY=18080 docker compose up -d` y cambia 8080 por 18080 en toda la guía.

## 3.2 · Comprueba que Redis y MongoDB contestan

```
# Le pregunta a Redis si está vivo
docker compose exec -T redis redis-cli ping
# Abre la consola de MongoDB en la base notificacion y cuenta los avisos guardados
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
```

**Qué deberías ver**

`PONG` y `0`. Redis está vivo y en Mongo todavía no hay avisos.

**Opcional:** si te gustan las interfaces gráficas, MongoDB Compass se conecta a `mongodb://localhost:27017` y RedisInsight a `localhost:6379`. No hacen falta: todo el lab se hace con `mongosh` y `redis-cli` dentro de los contenedores.

### 3.3 · Crea las cuentas de Ana, Beto y Carla

```
# Cada comando abre una cuenta por el gateway con su saldo inicial
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000003","titular":"Carla","saldoInicial":300}'
```

Cada alta da **201**. Si te da **409**, la cuenta ya existía de otro lab (el volumen de Postgres se conserva). Sirve igual, solo que tus saldos serán otros.

Haz ya una transferencia: es la que usarás en toda la guía, cambiando las CLABEs y el monto.

```
# Manda 200 de Ana a Beto; responde 201 con "estado":"COMPLETADA"
curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":200}'
```

### 3.4 · Mide: cada consulta va a Postgres

```
# Pide el saldo de Ana diez veces seguidas (echo agrega un salto de línea entre respuestas)
for i in $(seq 1 10); do
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
done

# Cuenta en el log de las tres copias de cuenta la línea que sale cada vez que se lee Postgres
docker compose logs cuenta | grep -c "leida de Postgres"

# Muestra cuántas lecturas acertó (hits) y falló (misses) Redis desde que arrancó
docker compose exec -T redis redis-cli INFO stats | grep keyspace
```

**Qué deberías ver**

El conteo de "leida de Postgres" da **10 o más**: las diez consultas, más las lecturas de la transferencia. En Redis, `keyspace_hits:0` y `keyspace_misses:0`: nadie lo usa todavía.

Fíjate en `leidaEn` de cada respuesta: cambia en las diez, porque las diez se leyeron de Postgres.

**Para la bitácora:** captura el conteo. Es tu "antes".

**Ya tengo mi "antes": diez consultas, diez lecturas a Postgres.**

# 4 La caché del saldo en Redis

Minutos 15 a 40 · Parte 1 · guiada, en el servicio `cuenta`.

## La idea: cache-aside

Vas a usar el patrón más común, **cache-aside**: la app pregunta primero a la caché y, si no está, va a la base y deja una copia para la próxima vez.




**Hit: estaba en Redis**

Contesta Redis. El método ni corre y Postgres ni se entera.

**Miss: no estaba**

Corre el método, lee Postgres y Spring guarda la copia con `SET ... PX 30000` (vive 30 s).

Y cuando el saldo cambia (un cargo o un abono), la copia se **borra** con `DEL`. No se actualiza: se borra, y la siguiente consulta la vuelve a llenar con el saldo nuevo.




El orden importa: primero el commit, después el borrado. Lo vemos en la pregunta 4 del cierre.

En Spring no escribes los `GET`, `SET` y `DEL` a mano: los hacen dos anotaciones. `@Cacheable` en el método que lee y `@CacheEvict` en los que cambian el saldo.

## 4.1 · Enciende y configura la caché

`cuenta/src/main/java/mx/mexibanco/cuenta/ConfiguracionCache.java`

**Qué hace este archivo:** Spring Boot ya arma solo un `RedisCacheManager`, porque el `pom.xml` trae `spring-boot-starter-data-redis`. Aquí solo lo enciendes y le cambias tres cosas a la caché `cuentas`.

**Qué cambias:** agrega `@EnableCaching` arriba de la clase y el `@Bean` de abajo. Lo verde es lo nuevo.

```java
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.autoconfigure.cache.RedisCacheManagerBuilderCustomizer;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.serializer.Jackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializationContext.SerializationPair;

import java.time.Duration;

// sin @EnableCaching, @Cacheable y @CacheEvict se ignoran sin avisar
@EnableCaching
@Configuration
public class ConfiguracionCache {

    public static final String CUENTAS = "cuentas";

    @Bean
    RedisCacheManagerBuilderCustomizer cacheDeCuentas(
            @Value("${mexibanco.cache.ttl:30s}") Duration ttl, ObjectMapper json) {
        RedisCacheConfiguration cuentas = RedisCacheConfiguration.defaultCacheConfig()
            // 1. cada entrada vive 30 s y luego Redis la borra sola
            .entryTtl(ttl)
            .disableCachingNullValues()
            // 2. el valor se guarda como JSON de CuentaVista, legible con redis-cli
            .serializeValuesWith(SerializationPair.fromSerializer(
                new Jackson2JsonRedisSerializer<>(json, CuentaVista.class)));
        return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
            // 3. el DEL de @CacheEvict espera al commit de la transacción
            .transactionAware();
    }
}
```

| LÍNEA | POR QUÉ |
| --- | --- |
| `entryTtl(ttl)` | Sin TTL, una entrada en Redis no caduca nunca. Si alguien cambia el saldo por fuera de la app (la Parte 4), el dato viejo se quedaría para siempre. |
| `Jackson2JsonRedisSerializer` | Por omisión Spring guarda el objeto con la serialización binaria de Java (`\xac\xed...`). En JSON lo puedes leer con `redis-cli GET`. |
| `transactionAware()` | El `DEL` de `@CacheEvict` espera a que la transacción haga commit. Si borrara antes, otra consulta podría leer el saldo viejo en esa ventana y volver a dejarlo en la caché. |

### 4.2 · Cachea la consulta que muestra

`cuenta/src/main/java/mx/mexibanco/cuenta/ConsultaDeSaldo.java`

**Qué hace este archivo:** contesta `GET /cuentas/{clabe}`, lo que ve el cliente en la app. Cada vez que corre, escribe en el log `[consulta] ... leida de Postgres`.

**Qué cambias:** una anotación arriba de `consultar`. La `key` dice con qué se arma la clave en Redis: `#clabe` es el parámetro, y la clave queda `cuentas::002180000000000001`.

```
import org.springframework.cache.annotation.Cacheable;
...
    @Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
    public CuentaVista consultar(String clabe) {
        log.info("[consulta] {} leida de Postgres", clabe);   // en un hit, esta línea ya no
sale
        return cuentas.findByClabe(clabe)
            .map(CuentaVista::de)
            .orElseThrow(() -> new NoEncontradoException("No existe la CLABE " + clabe));
    }
```

### 4.3 · Borra la copia cuando cambia el saldo

`cuenta/src/main/java/mx/mexibanco/cuenta/CuentaService.java`

**Qué hace este archivo:** las reglas de la cuenta. `cargar` revisa que alcance el saldo y lo resta; `abonar` lo suma. Los dos los llama la saga de transferencia por HTTP.

**Qué cambias:** la misma anotación arriba de `cargar` y de `abonar`, junto al `@Transactional` que ya tienen.

```
import org.springframework.cache.annotation.CacheEvict;
...
    @CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
    @Transactional
    public SaldoAplicado cargar(String clabe, BigDecimal monto, String clave) { ... }

    @CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
    @Transactional
    public SaldoAplicado abonar(String clabe, BigDecimal monto, String clave) { ... }
```

**Lo que NO tocas: CuentaService.consultar**

En el mismo archivo hay otro `consultar`. Es la lectura que usa `cargar` para **decidir** si alcanza el saldo, y siempre va a Postgres. Si la cachearas, un cargo podría aprobarse con un saldo de hace 30 s.

Por eso la consulta que muestra vive en otra clase, `ConsultaDeSaldo`. Hay una segunda razón: si pones `@Cacheable` en un método y lo llamas desde su **misma** clase, la llamada no pasa por el proxy de Spring y la anotación se ignora sin avisar.

## 4.4 · Reconstruye cuenta y comprueba

```
# Vuelve a construir solo la imagen de cuenta y reemplaza sus tres copias
docker compose up --build -d cuenta
```

**Antes de correrlo, predice:** vas a pedir el saldo de Ana seis veces y van a contestar distintas copias de cuenta. ¿Cuántas veces se va a leer Postgres?

Ver la respuesta

```
# Pide el saldo de Ana seis veces. -D - muestra también las cabeceras;
# grep deja solo qué copia contestó (X-Instancia) y el cuerpo con leidaEn
for i in 1 2 3 4 5 6; do
  curl -s -D - http://localhost:8080/api/cuentas/0021800000000000001 | grep -iE 'x-
instancia|leidaEn'
done

# Cuenta cuántas veces se leyó Postgres en los últimos 30 segundos
docker compose logs --since 30s cuenta | grep -c "leida de Postgres"

# Muestra la copia guardada en Redis y cuántos segundos le quedan
docker compose exec -T redis redis-cli GET cuentas::0021800000000000001
docker compose exec -T redis redis-cli TTL cuentas::0021800000000000001
```

**Qué deberías ver**

```
X-Instancia: e22ac7714e7c
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-10-
31T16:21:41.883Z"}
X-Instancia: 77217fc575f4  { ... el mismo leidaEn ... }
X-Instancia: c99a903157ed  { ... el mismo leidaEn ... }
...
1                                                  ← una sola lectura a Postgres
"
{\"clabe\":\"002180000000000001\",\"titular\":\"Ana\",\"saldo\":800.00,\"leidaEn\":\"2026
-10-31T16:21:41.883Z\"}"
(integer) 29
```

Tus `X-Instancia`, saldos y horas serán otros. Lo que importa: la instancia cambia, `leidaEn` no.

## 4.5 · Comprueba que la copia se borra

Abre una **segunda terminal** y deja corriendo esto: muestra cada comando que recibe Redis, en vivo.

```
# Terminal 2: espía de Redis (Ctrl+C para salir)
docker compose exec -T redis redis-cli MONITOR
```

En la primera terminal, transfiere 10 de Ana a Beto y revisa si la copia sigue ahí:

```
# Transfiere 10 de Ana a Beto
curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10}'
# ¿Existe la copia de Ana? 1 = sí, 0 = no
docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001
# Vuelve a pedir el saldo: debe traer el saldo nuevo y un leidaEn nuevo
curl -s http://localhost:8080/api/cuentas/002180000000000001
```

**Qué deberías ver en el MONITOR, en este orden**

```
"GET" "cuentas::002180000000000001"        ← la saga revisa que exista la cuenta
"DEL" "cuentas::002180000000000001"        ← después del commit del cargo
"DEL" "cuentas::002180000000000002"        ← después del commit del abono a Beto
"GET" "cuentas::002180000000000001"        ← tu consulta: miss
"SET" "cuentas::002180000000000001" "{...\"saldo\":790.00...}" "PX" "30000"
```

Y `EXISTS` da `0`. Puede haber un `GET` y `SET` de Beto entre medio: son las comprobaciones de la saga.

**Las tres copias comparten la caché, veo el JSON con su TTL y la copia se borra al transferir.**

### 4.6 · Mide: ¿qué compraste con la caché?

**Hit ratio.** Redis cuenta sus aciertos ( `keyspace_hits` ) y sus fallos ( `keyspace_misses` ). El hit ratio es `hits / (hits + misses)` .

```
# Pone en cero los contadores de Redis
docker compose exec -T redis redis-cli CONFIG RESETSTAT
# Escenario A: cien consultas seguidas del saldo de Ana (-o /dev/null tira la respuesta)
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
done
# Muestra los contadores
docker compose exec -T redis redis-cli INFO stats | grep keyspace
```

**Escenario B:** lo mismo, pero con una transferencia de 1 peso de Ana a Beto cada diez consultas. Explica por qué baja. Ojo: tus propios `redis-cli GET` también cuentan.

**Lo que dio en la prueba del profe**

A: 99 hits y 1 miss, **99 %**. B: 101 hits y 19 misses, **84 %**. Cada transferencia borra a Ana y a Beto, y la siguiente lectura de cada uno falla.

**Latencia.** Este comando hace 100 consultas, guarda el tiempo de cada una y calcula promedio, mediana y p95 en milisegundos:

```
# -w '%{time_total}' imprime cuántos segundos tardó cada consulta
for i in $(seq 1 100); do
  curl -s -o /dev/null -w '%{time_total}\n'
http://localhost:8080/api/cuentas/002180000000000001
done \
# sort los ordena; awk los pasa a ms y saca promedio, mediana (el de en medio) y p95
  | sort -n \
  | awk '{t[NR]=$1*1000; s+=$1*1000}
         END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)],
t[int(NR*0.95)]}'
```

Para medir **sin** caché no tocas código: `application.yml` ya lee la variable `CACHE_TIPO` .

```
# Reinicia cuenta con la caché apagada; espera a que esté healthy con docker compose ps
CACHE_TIPO=none docker compose up -d cuenta
# ... mide igual, y cuenta también las lecturas a Postgres ...
# Regresa cuenta con la caché encendida
docker compose up -d cuenta
```

**Predice:** ¿cuánto más rápida será la consulta con caché?

**Ver lo que pasó en la prueba**

**Para la bitácora:** corre cada medición dos o tres veces y reporta todas, con y sin caché, junto con las lecturas a Postgres de cada caso.

**Tengo hit ratio de A y B, y latencia y lecturas a Postgres con y sin caché.**

## 5 Los avisos en MongoDB

Minutos 40 a 65 · Parte 2 · en el servicio `notificacion`.

## La idea: un documento por aviso, con la forma de su canal

Cada transferencia genera **dos** avisos: uno para quien envía y otro para quien recibe. Cada aviso es un **documento**: campos comunes a todos, más los de su canal. Nadie hace `ALTER TABLE` para que existan: el esquema lo aplica quien lee.

```
// Ana · SMS
{
  _id: "41bea4d1-...:0021...001",
  eventoId: "41bea4d1-...",
  transferenciaId: 1,
  clabe: "002180000000000001",
  tipo: "TRANSFERENCIA_ENVIADA",
  monto: Decimal128("200"),
  saldo: Decimal128("800.00"),
  fecha: ISODate("2026-10-31..."),
  canal: "SMS",
  telefono: "+52 55 5555 0101",
  texto: "...por 200..."
}
```

```
// Beto · CORREO
{
  _id: "41bea4d1-...:0021...002",
  eventoId: "41bea4d1-...",
  transferenciaId: 1,
  clabe: "002180000000000002",
  tipo: "TRANSFERENCIA_RECIBIDA",
  monto: Decimal128("200"),
  saldo: Decimal128("700.00"),
  fecha: ISODate("2026-10-31..."),
  canal: "CORREO",
  correo: "beto@correo.mx",
  asunto: "Recibiste...",
  cuerpo: "...por 200..."
}
```

```
// Carla · PUSH
{
  _id: "efa3b04e-...:0021...003",
  eventoId: "efa3b04e-...",
  transferenciaId: 2,
  clabe: "002180000000000003",
  tipo: "TRANSFERENCIA_RECIBIDA",
  monto: Decimal128("50"),
  saldo: Decimal128("350.00"),
  fecha: ISODate("2026-10-31..."),
  canal: "PUSH",
  token: "fcm:demo-0021...003",
  titulo: "Recibiste...",
  cuerpo: "...por 50..."
}
```

Lo de arriba es igual en los tres; lo de abajo, en negritas, depende del canal.

Dos decisiones del contrato que vas a programar:

- **El `_id` lo pone el banco, no Mongo:** `eventoId:clabe`. El mismo evento con la misma CLABE da siempre el mismo `_id`, y Mongo no deja guardar dos documentos con el mismo `_id`. Así un evento repetido no duplica el aviso (la Parte 3). No es solo el `eventoId` porque un evento genera dos avisos.
- **El dinero va en `Decimal128`**, nunca en `double`: `0.1 + 0.2` en punto flotante no da `0.3`.

## El contrato

Usa **exactamente** estos nombres. Así tus consultas de `mongosh` dan lo mismo que las de tus compañeros y así se revisa tu entrega.

| PIEZA | VALOR |
|---|---|
| Base y colección | Base `notificacion`, colección `avisos` (constante `COLECCION`) |
| `_id` | `eventoId + ":" + clabe` |
| Campos de todos | `eventoId`, `transferenciaId`, `clabe`, `tipo`, `monto` y `saldo` (Decimal128), `fecha` (la del evento), `canal` |
| Canal `SMS` | `telefono`, `texto` |
| Canal `CORREO` | `correo`, `asunto`, `cuerpo` |
| Canal `PUSH` | `token`, `titulo`, `cuerpo` |
| Índice | `clabe_fecha`: `{ clabe: 1, fecha: -1 }` |

| | |
|---|---|
| Endpoint | `GET /api/notificaciones/{clabe}`: los últimos 10, del más nuevo al más viejo |

`notificacion/src/main/java/mx/mexibanco/notificacion/avisos/BitacoraDeAvisos.java`

**Qué hace este archivo:** todo lo de MongoDB en notificación. Ya trae resuelto a quién se avisa (`destinatarios`), el texto de cada aviso (`texto`) y por qué canal (`DirectorioDeContactos`). Usa `MongoTemplate`, que es a Mongo lo que `JdbcTemplate` es a SQL. Tú llenas cuatro métodos. Agrega primero estos imports:

```
import org.bson.types.Decimal128;
import org.springframework.dao.DuplicateKeyException;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.index.Index;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.data.mongodb.core.query.Query;
import java.util.Date;
```

### 5.1 · `documento(...)`: arma el aviso

**Qué hace:** convierte un evento y un destinatario en un `org.bson.Document`, que es un mapa de campo a valor. Primero los campos comunes, después un `switch` sobre el `Contacto` que agrega los del canal.

```
public static Document documento(EventoTransferencia evento, Destinatario destinatario,
Contacto contacto) {
    Document aviso = new Document("_id", evento.eventoId() + ":" + destinatario.clabe())
        .append("eventoId", evento.eventoId().toString())
        .append("transferenciaId", evento.transferenciaId())
        .append("clabe", destinatario.clabe())
        .append("tipo", destinatario.tipo())
        .append("monto", new Decimal128(evento.monto()))       // dinero: Decimal128, nunca
double
        .append("saldo", new Decimal128(destinatario.saldo()))
        .append("fecha", Date.from(evento.fecha()));           // la fecha del evento, no la
de hoy
    String texto = texto(destinatario, evento);
    switch (contacto) {
        case Contacto.Sms sms -> aviso
            .append("canal", "SMS")
            .append("telefono", sms.telefono())
            .append("texto", texto);
        // TU TURNO: el caso Contacto.Correo (canal, correo, asunto, cuerpo)
        //           y el caso Contacto.Push (canal, token, titulo, cuerpo)
    }
    return aviso;
}
```

Pista 1: ¿por qué no compila mi switch?

Pista 2: ¿qué pongo en asunto y titulo?

Ver los dos casos que faltan

## 5.2 · `registrar(evento)` : guarda sin duplicar

**Qué hace:** por cada destinatario arma su documento y lo inserta. Regresa cuántos eran **nuevos**. Si Mongo contesta que el `_id` ya existe (`DuplicateKeyException`), ese aviso ya estaba: no es un error, no se cuenta y se sigue con el siguiente.

```
public int registrar(EventoTransferencia evento) {
    int nuevos = 0;
    for (Destinatario destinatario : destinatarios(evento)) {
        Document aviso = documento(evento, destinatario, contactos.de(destinatario.clabe()));
        try {
            mongo.insert(aviso, COLECCION);
            nuevos++;
        } catch (DuplicateKeyException yaEstaba) {
            log.info("[avisos] {} ya estaba: evento repetido, no se duplica",
aviso.get("_id"));
        }
    }
    return nuevos;
}
```

**¿Por qué uno por uno y no todos juntos?** Un insert de varios se detiene en el primer `_id` repetido. Si el proceso se cayó después de guardar el primer aviso, la reentrega tiene que poder guardar el segundo.

## 5.3 · `crearIndice()` y `ultimos(clabe)` : leer rápido

**Qué hacen:** `ultimos` busca los avisos de una CLABE, del más nuevo al más viejo, y se queda con 10. `crearIndice` corre una sola vez al arrancar (por el `@EventListener` que ya tiene) y crea el índice que sigue la forma de esa consulta: primero por `clabe` y luego por `fecha`. Sin el índice, Mongo recorre toda la colección y ordena en memoria.

```
public void crearIndice() {
    String nombre = mongo.indexOps(COLECCION).ensureIndex(
        new Index().on("clabe", Sort.Direction.ASC).on("fecha",
Sort.Direction.DESC).named("clabe_fecha"));
    log.info("[avisos] indice {} listo en {}", nombre, COLECCION);
}

public List<Document> ultimos(String clabe) {
    // TU TURNO: una Query con clabe igual a la que llega, ordenada por fecha descendente y
limit(ULTIMOS)
    //           y luego mongo.find(consulta, Document.class, COLECCION)
}
```

**Pista: ¿cómo se escribe la Query?**

**Ver el método completo**

## 5.4 · Conecta el consumidor (una línea)

`notificacion/src/main/java/mx/mexibanco/notificacion/mensajeria/`**`ConsumidorDeTransferencias.java`**

**Qué hace este archivo:** es el consumidor de RabbitMQ de la S09. Recibe cada `TransferenciaRealizada`, guarda los avisos y después le confirma al broker con `basicAck`. Ya trae `BitacoraDeAvisos` inyectada en el constructor, sin usarla.

**Qué cambias:** busca el `TODO S11` dentro de `recibir` y cambia la línea que guarda:

```
            nuevos = notificaciones.avisarDeTransferencia(evento).size();   // antes: bandeja
de Postgres
            nuevos = bitacora.registrar(evento);                            // ahora:
colección avisos de MongoDB
```

**Solo en Mongo, no en los dos**

Guardar en Postgres **y** en Mongo sería una doble escritura sin transacción común: si una falla, quedan distintas. Es el problema que resolvió el Outbox en la S09. Los avisos se **mudan**, no se copian.

### 5.5 · Reconstruye notificación y comprueba

```
# Vuelve a construir solo la imagen de notificacion y la reemplaza
docker compose up --build -d notificacion
# Busca en su log la línea del índice
docker compose logs notificacion | grep avisos
```

**Qué deberías ver**

`[avisos] indice clabe_fecha listo en avisos`

**Las tres formas.** Haz una transferencia de Ana a Beto y otra de Ana a Carla (el comando de 3.3, cambiando la CLABE destino a `...003`). Luego:

```
# Muestra todos los avisos, del más viejo al más nuevo
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
# Muestra solo los que tienen teléfono, con tres campos: el esquema lo aplica quien lee
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists:
true}}, {clabe: 1, canal: 1, telefono: 1})'
```

**Qué deberías ver**

Cuatro avisos con las formas de la sección 5: dos SMS de Ana, un CORREO de Beto y un PUSH de Carla. El segundo comando solo trae los dos de Ana.

**Los últimos 10.** Haz doce transferencias pequeñas de Ana a Beto (de 1 peso) y pide la bitácora de Beto:

```
# Doce transferencias de 1 peso de Ana a Beto
for i in $(seq 1 12); do curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias
-H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"002180000000000002","monto":1}';
done
# La bitácora de avisos de Beto por el gateway
curl -s http://localhost:8080/api/notificaciones/0021800000000000002
```

**Qué deberías ver**

Exactamente **10** avisos, todos `CORREO`, del más reciente al más viejo.

**El índice trabaja.** `explain` le pide a Mongo que diga **cómo** resolvió la consulta, sin cambiar nada:

```
# Lista los índices; debe aparecer clabe_fecha además de _id_
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
# El plan de la consulta de ultimos(): busca la etapa IXSCAN y totalDocsExamined
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe:
"002180000000000002"}).sort({fecha: -1}).limit(10).explain("executionStats")'
# Lo mismo, pero obligando a Mongo a NO usar el índice ($natural = recorrer en orden de disco)
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe:
"002180000000000002"}).sort({fecha: -1}).limit(10).hint({$natural:
1}).explain("executionStats")'
```

**Lo que dio en la prueba del profe, con 28 avisos**

| | ETAPAS | DOCUMENTOS EXAMINADOS |
|---|---|---|
| Con índice | `LIMIT ← FETCH ← IXSCAN` | **10** |
| Sin índice | `SORT ← COLLSCAN` | **28**, todos |

Con 28 avisos la diferencia es chica. Con 28 millones, es la diferencia entre leer 10 y leer 28 millones.

**Veo las tres formas, el endpoint da 10 y tengo los dos explain.**

## 6 Rompe algo: el evento duplicado

5 minutos · Parte 3 · en clase o de tarea.

La mensajería de la S09 entrega **al menos una vez**: un evento puede llegar dos veces. Provócalo publicando el mismo evento dos veces desde la API del panel de RabbitMQ. Usa CLABEs inventadas (`...091` y `...092`) para no mezclarlo con tus cuentas.

```
# Publica un evento fijo en el exchange transferencias. Córrelo DOS veces.
curl -u mexibanco:mexibanco -X POST
http://localhost:15672/api/exchanges/%2F/transferencias/publish \
  -H 'Content-Type: application/json' \
  -d '{"properties":
{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada",
"payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-
555555555555\",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen\":\"
002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,
\"saldoDestino\":525,\"fecha\":\"2026-10-31T13:00:00Z\"}"}'

# Cuenta los avisos de ese evento
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments({eventoId:
"11111111-2222-4333-8444-555555555555"})'
# Busca en el log de notificación lo que pasó con el segundo
docker compose logs notificacion | grep "ya estaba"
```

**Predice:** dos eventos iguales, dos avisos cada uno. ¿Cuántos documentos quedan?

Ver la respuesta

**Qué deberías ver**

```
{"routed":true}
{"routed":true}
2
[avisos] 11111111-...:002180000000000091 ya estaba: evento repetido, no se duplica
[avisos] 11111111-...:002180000000000092 ya estaba: evento repetido, no se duplica
```

**Otra forma, como en la S09:** levanta notificación con `PAUSA_ANTES_DEL_ACK_MS=15000 docker compose up -d notificacion`, haz una transferencia, mátalo durante la pausa con `docker compose kill notificacion` y vuelve a levantarlo sin pausa con `docker compose up -d notificacion`. El mensaje regresa con `redelivered=true` y no se duplica nada.

**El mismo evento dos veces deja dos avisos y el log dice "ya estaba".**

## 7 Rompe algo: el dato viejo

10 minutos · Parte 4 · en clase o de tarea.

La caché puede mentir. Provócalo cambiando el saldo en Postgres **por fuera** de la app, como haría un script mal hecho: así no pasa por el `@CacheEvict` y nadie borra la copia.

Redis: el saldo de antes
la copia, con TTL

≠

Postgres: 5
la verdad

```
# 1. Consulta a Ana: su saldo queda en la caché. Anota cuánto tiene.
curl -s http://localhost:8080/api/cuentas/002180000000000001
# 2. Cambia el saldo directo en Postgres (psql es la consola de Postgres)
docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 5 where
clabe = '002180000000000001'"
# 3. Cada 5 segundos: cuánto TTL le queda a la copia y qué saldo ve el cliente
for i in $(seq 1 8); do
  docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
  sleep 5
done
```

En **otra terminal**, mientras el saldo viejo sigue en la caché, intenta transferir 100 de Ana a Beto:

```
curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":100}'
```

**Predice:** la app todavía le muestra a Ana su saldo de antes, de cientos de pesos. ¿La transferencia de 100 pasa?

**Ver lo que pasó en la prueba**

**Lo que dio en la prueba del profe**

```
TTL 29  saldo 652.00  leidaEn 21:27:04
TTL 24  saldo 652.00
...
TTL 3   saldo 652.00
TTL -2  saldo 5.00    leidaEn 21:27:36     ← caducó, se leyó Postgres
```

El cliente vio un saldo que ya no existía durante **32 segundos**.

**Al terminar, regresa el saldo de Ana**

Corre otra vez el paso 2 con el saldo que tenía Ana. Este `UPDATE` rompe a propósito el cuadre entre el saldo y los movimientos.

**Sé cuántos segundos se vio el saldo viejo y qué contestó la transferencia.**

## 8 Reto: Redis se cae

Opcional, suma puntos · de tarea.

```
# Apaga solo Redis; todo lo demás sigue arriba
docker compose stop redis
```

Prueba un GET del saldo y una transferencia. En tu versión, ¿qué pasa? La caché era solo una copia para mostrar: ¿debería poder tumbar algo?

Haz que cuenta siga funcionando sin Redis: el GET va a Postgres (más lento) y la transferencia se completa. Antes de elegir dónde atrapar el error, piensa **cuándo** ocurre el `DEL` con `transactionAware()`, y qué le pasaría a la saga si cuenta contesta 500 a un cargo que **ya se confirmó**.

**Pista 1**

**Pista 2**

**Terminado cuando:** con Redis apagado, el GET responde 200, la transferencia responde 201, los saldos cuadran y el log dice qué operación de caché falló. Al encender Redis (`docker compose start redis`), todo sigue.

## 9 Qué entregas

En tu rama `entregas_apellido_nombre` del repo del grupo:

```
entregas/apellido_nombre/s11/
├── bitacora-s11.pdf
└── codigo/
    ├── cuenta/              ← la carpeta completa del servicio, SIN target/
    ├── notificacion/        ← la carpeta completa del servicio, SIN target/
    └── docker-compose.yml   ← solo si lo cambiaste
```

**La bitácora**, en PDF y con capturas de cada parte:

1. **El punto de partida:** diez consultas, diez lecturas a Postgres (sección 3.4).
2. **La caché:** tu código explicado con tus palabras, las tres copias compartiendo la caché, el JSON y el TTL en Redis, y la copia borrada después de una transferencia.
3. **Tus medidas:** hit ratio de A y B, latencia con y sin caché (todas las corridas) y lecturas a Postgres en cada caso.
4. **Los avisos:** las tres formas en `mongosh`, el endpoint con 10, el índice y los dos `explain`.
5. **El duplicado** y **el dato viejo**.

6. **Las preguntas de cierre** (abajo).
7. **Redis caído,** si hiciste el reto.

```
git checkout entregas_apellido_nombre          # cámbiate a tu rama de entregas
git pull origin main                            # trae lo último del repo del grupo
mkdir -p entregas/apellido_nombre/s11/codigo    # crea la carpeta de esta entrega
# ... copia cuenta/ y notificacion/ (sin target/) y tu bitacora-s11.pdf ...
git add entregas/apellido_nombre/s11            # marca la carpeta para el commit
git commit -m "Lab S11: persistencia políglota y caché"
git push                                        # súbelo: el push ES la entrega
```

**No abras pull request.** Tarde no baja puntos, pero queda registrado en tu tendencia (ver `ENTREGAS.md`).

### Preguntas de cierre

Respóndelas en la bitácora, cortas y con tus palabras:

1. ¿Por qué MongoDB para los avisos y **no** para el saldo? Usa lo que viste: la forma de cada aviso, cómo se leen y qué tendría que garantizar el saldo.
2. ¿Qué hit ratio mediste en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿qué compraste con la caché, velocidad o capacidad?
3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché? Usa lo que viste en la Parte 4.
4. Al aplicar un movimiento, ¿por qué se hace `DEL` de la copia y no `SET` con el saldo nuevo? ¿Y por qué después del commit y no antes?

## 10 Si te atoras

| SÍNTOMA | CAUSA PROBABLE Y ARREGLO |
|---|---|
| Cada GET sigue saliendo como "leida de Postgres" | Falta `@EnableCaching` en `ConfiguracionCache`, o pusiste `@Cacheable` en `CuentaService` en lugar de `ConsultaDeSaldo`. ¿Reconstruiste cuenta? |
| En Redis el valor sale con `\xac\xed...` | Falta el serializador JSON: Spring usó la serialización binaria de Java. |
| `SerializationException`, o el GET da 500 desde la segunda vez | Revisa que el serializador sea de `CuentaVista.class` y que uses el `ObjectMapper` que inyecta Spring (ese ya sabe escribir fechas). |
| Después de transferir, `EXISTS` sigue dando 1 | Falta `@CacheEvict` en `cargar` o en `abonar`, o la `key` no es `"#clabe"`. |

| | |
|---|---|
| El índice no aparece | `crearIndice()` corre al arrancar. Revisa el log: `docker compose logs notificacion \| grep avisos`. |
| En Mongo `monto` sale como texto o como `Double` | Usa `new Decimal128(...)` con el `BigDecimal`. |
| `UnsupportedOperationException: TODO S11` en el log de notificación | Te falta llenar un método de `BitacoraDeAvisos`: el mensaje dice cuál. |
| El build de Docker falla | Lee el primer `ERROR` del build: casi siempre es un import que falta. Si cambiaste el constructor del consumidor, regrésalo: ya trae `BitacoraDeAvisos`. |
| `./mvnw test` falla en `ConsumidorDeTransferenciasTest` | Es esperado: esa prueba todavía revisa la bandeja de Postgres. Docker no corre las pruebas, así que no te bloquea. |
| **Windows:** `grep`, `awk` o `seq` "no se reconoce como comando" | Estás en PowerShell o CMD. Abre **Git Bash** (ver el recuadro del inicio). |
| **Windows:** `the input device is not a TTY` | Le falta `-T` al `docker compose exec`. Todos los de la guía lo traen; si escribiste uno a mano, agrégalo. |
| **Windows:** el build falla con `/bin/sh^M` o `./mvnw: not found` | El `mvnw` quedó con finales de línea de Windows. La rama ya trae un `.gitattributes` que lo evita en un clon nuevo; si clonaste antes, en Git Bash, dentro del repo, corre `for f in */mvnw; do sed -i 's/\r$//' "$f"; done`. Le quita el final de línea de Windows a los siete `mvnw` y no toca nada más. |
| Quiero empezar de cero con Mongo | `docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.drop()'` borra la colección; el índice se crea otra vez al reiniciar notificación. |

Si cambias un solo servicio, reconstruye solo ese: `docker compose up --build -d cuenta` o `... notificacion`. Dudas o errores: abre un Issue en el repo del grupo con el error completo y el comando que corriste.